Amazon SDE Intern — Behavioral Prep

Mahip Parekh

STORIES

A — Automation Script (IDX)

Use for: Ownership, Bias for Action, Invent and Simplify, Frugality, Deliver Results

Situation: My team was asked to produce a report showing what software versions each of our 900+ client websites were running across dev, staging, and production environments.

Task: The only way to check was to manually log into each website's backend and note down the versions. At that scale, doing it by hand would have taken forever and been error prone.

Action: I proposed automating it. I built a script in Java using Selenium that logged into each website, pulled the version info, and dumped everything into an Excel sheet. Halfway through, I discovered a bunch of old websites that were completely dead but nobody knew about. I flagged this to my manager and he realized we were getting two things done at once, the version report and a cleanup of dead websites.

Result: I delivered version details for every active website. 400+ dev, 300+ staging, 270+ production. The report was documented on Confluence, pushed to Bitbucket, and other teams asked to use it for their own purposes.

Follow ups:

- Why Java/Selenium → comfortable with Java, Selenium was right for browser automation
- Hardest part → handling dead URLs and edge cases across 900+ sites
- What would you do differently → batch Excel writes in memory instead of row by row, host on a server instead of running on my laptop for a week
- What happened after → I optimized data insertion to bulk writes, my team lead handled the hosting side

---

B — Testing Disagreement (Calendar Project)

Use for: Disagree and Commit, Deliver Results

Situation: For the final phase of our semester long calendar project, we had to add several new features and build a completely new GUI. Our target was 100% mutation test coverage.

Task: My teammate and I disagreed on how to handle testing. He wanted to implement everything first and test at the end. I was worried that cramming all the testing at the end would result in weak tests since mutation coverage requires really thorough test writing.

Action: I pushed back on his approach, and then suggested a middle ground. We'd test each feature's core behavior as we built it and save edge case testing for later. We made a checklist, split the work, and rotated between implementing and testing.

Result: We finished before the deadline with 100% functionality and 98% mutation test coverage. The approach worked because we addressed both concerns.

Follow ups:

- How we rotated roles → I'd implement 2-3 features, he'd write tests for them, then we'd swap
- What mutation testing means → tests need to be strong enough to catch intentional code mutations
- What I'd do differently → suggest the compromise sooner instead of going back and forth
- What did you learn → stop debating the approach and align on what matters most first. Once we agreed both speed and quality were non negotiable, the compromise came naturally

---

C — Client Email Escalation (IDX)

Use for: Earn Trust, Dive Deep

Situation: After a client website upgrade, it was my responsibility to go through a QA checklist before giving the green flag to the hosting team to map the live URL. During these checks, I found a UI bug where the publish button wasn't working.

Task: Before testing, I disabled the client email notification toggle in the backend so the client wouldn't get alerts while I debugged. But the site's cache was stale, so my toggle change never took effect.

Action: Every click I made fired an email to the actual client. It got escalated. Before the meeting with the client's manager, I sat with my team and figured out the root cause. In the meeting, I explained exactly what happened and gave a live demo showing how stale cache causes this behavior.

Result: The client was satisfied with the explanation. My manager backed me up because I came prepared and was honest. I learned to always verify my changes are reflected, not just assume they took effect.

Follow ups:

- How did you feel → panicked at first, but knew hiding it would be worse

- Did your manager react negatively → no, because I came with the problem and the root cause already figured out
- Why the live demo → wanted the client manager to see the issue, not just hear me explain it
- What changed after → I never assume a change took effect. I always verify, especially on live environments

---

D — Not Asking for Help (IDX)

Use for: Learn and Be Curious, Weakness

Situation: This was my first job out of college. I joined the product support team and was given access to the codebase to ramp up. I had this mindset that I should figure out everything myself.

Task: I spent two weeks going through the code alone, Googling everything, never once asking my teammates who were sitting right next to me.

Action: After two weeks, my manager told me I wasn't learning fast enough. He knew exactly why. He addressed it supportively, saying the team has my back and I should use them. I changed my approach completely. I started reaching out to my senior manager, walking over to his desk, and having real conversations about how the product worked.

Result: My ramp up accelerated immediately. I learned things I hadn't even discovered on my own. It taught me that asking for help isn't a weakness. It's how you learn faster and build relationships.

Follow ups:

- Do you still struggle with this → my instinct is still there but I set a 20-30 min timer. If I'm stuck, I reach out
- Why that mindset → college is individual work, you're graded on what you know. Professional work is about team output
- How did you feel → embarrassed, but he said it supportively so I absorbed it

---

E — Distributed File System (Northeastern)

Use for: Invent and Simplify, Learn and Be Curious, Dive Deep

Situation: For my distributed systems course, my partner and I had to build a file storage system from scratch in C++. I knew concepts like consistent hashing and chain replication from class but had never implemented any of them. Zero experience with TCP socket programming.

Task: We needed to build a system that splits files into chunks, distributes them across storage nodes, handles concurrent clients, and survives node failures.

Action: I took on building the networking foundation. The raw TCP code got messy fast, so I refactored into clean wrapper classes. The hardest bug was messages not deserializing correctly between server and client. I realized there was no way to tell where one message ended and the next began. I fixed it by prepending every message with its length so the receiver always knows how many bytes to read.

Result: The final system handled 128+ concurrent connections with sub 200ms latency and 99.5% availability during node failures. We built automated tests that killed nodes mid operation and verified file recovery from replicas.

Follow ups:

- Consistent hashing → nodes and chunks placed on a virtual ring. Each chunk goes to nearest node clockwise. If a node dies, only nearby chunks redistribute
- Chain replication → writes go Head to Mid to Tail, acknowledged only at Tail. Reads from Tail only for latest state
- Work split → I built networking, partner planned architecture, then we both worked across layers
- What I'd do differently → add persistent storage like RocksDB, improve configuration to avoid hardcoded IPs
- Why C++ → course requirement

F — SSO Access Management (IDX)

Use for: Insist on Highest Standards

Situation: At IDX, I was responsible for managing SSO access to client environments. This was our team's KPI.

Task: People from other teams would regularly come to me saying "I already talked to your manager, just give me access" trying to skip the process.

Action: I always respectfully said no and asked them to raise a JIRA ticket first. Every single time. No exceptions. I explained my reasoning, that I needed to maintain a record of who has access to what site for what project.

Result: It maintained a proper paper trail for who has access to production environments. My manager supported this approach because accountability mattered more than convenience.

Follow ups:

- Why it mattered → production environment access is serious, you need audit trails
- Did anyone get upset → some were annoyed but once I explained the reason, they understood
- Did your manager back you → yes, fully

H — Coding Community Contest (Undergrad)

Use for: Ownership, Think Big

Situation: My professor noticed my interest in competitive programming and asked me to lead a coding community in college as Programming Head.

Task: Beyond regular workshops, I took on organizing a full competitive programming contest as part of our college's annual tech fest.

Action: I designed the problems myself at varying difficulty levels, wrote test cases, hosted the contest on CodeChef, and coordinated with lab faculty to reserve an entire floor of computers. During the contest, many students were doing CP for the first time. They knew the logic but couldn't figure out how to submit on the platform. I quickly set up volunteers to guide them through it.

Result: We had 60-70 participants from multiple colleges across the region. The contest was a hit and became a recurring part of the tech fest.

Follow ups:

- How problems were designed → easy for beginners, medium for intermediate, hard for competitive programmers. Tested and validated each one myself before uploading to CodeChef
- Promotion → college had teams that traveled to other campuses to promote the fest
- Hardest part → during the contest itself. Many first timers couldn't submit code. Had to stay on feet the entire time and set up volunteers on the fly
- How many participants → 60-70. Reserved entire floor of computer lab since students in India don't typically bring laptops
- What I'd do differently → run a 10 min platform walkthrough before the contest starts

---

I — Coding Community Events (Undergrad)

Use for: Customer Obsession

Situation: As Programming Head of our college's coding community, I was responsible for organizing technical events for students.

Task: Instead of just picking topics we thought were interesting, I wanted to understand what students actually needed. So I spent time talking to juniors to figure out where they were struggling.

Action: Turns out most of them weren't stuck on concepts. They were stuck on practical things. How to start with competitive programming. How to turn their thinking into actual code, whether it's a DSA problem or a project. Even non technical stuff like how to build a professional LinkedIn profile. We designed our entire event calendar around these needs. Every session came from listening first, not assuming.

Result: Attendance grew because students felt the events were relevant to them, not just generic tech talks. It taught me that the best way to serve people is to ask them what they need first, not assume you already know.

Follow ups:

- How did you gather feedback → informal conversations with juniors, asking them directly what they wish they knew
- How you prioritized topics → based on how many students mentioned the same pain point
- Most popular event → "how to start CP" and LinkedIn profile workshop, immediate practical value

---

J — Distributed Systems Teammate (Northeastern)

Use for: Are Right A Lot, Bias for Action, Deliver Results

Situation: For my distributed systems project, my teammate and I had very different working styles. I prefer to plan things out before building. My teammate was more of a figure it out on the fly kind of person. He wasn't prioritizing our project discussions because he had other coursework and kept saying we'd pull it off in the last week.

Task: I disagreed. We were building a distributed file storage system from scratch in C++. This wasn't something you could wing in a week. I believed we needed to plan the architecture early or we'd run into serious problems later.

Action: Instead of pressuring him to fix formal meetings, I adapted. I'd call him casually and ask how he'd want to implement a specific feature or what kind of fault tolerance he'd prefer. I collected his ideas over time. So by the time we finally sat down together, I already had draft action plans, possible approaches, and designs ready to discuss. Once we aligned, I'd go ahead and set up the skeleton before our next meeting. Interfaces, folder structures, basic assumptions, and starter tests. So every time we worked together, we had a solid path to follow instead of figuring things out from scratch.

Result: We completed the project on time with a system that handled 128+ concurrent connections and 99.5% availability during node failures. The early planning paid off. We also had really healthy knowledge exchange because of this style. My instinct to plan ahead instead of cramming at the end turned out to be the right call.

Follow ups:

- Why not force formal meetings → realized it wouldn't work with his style, so I adapted to get the same outcome
- What if he was right about last week → for a project this complex with multiple layers, that wouldn't have worked. We'd have been rewriting code constantly
- What did you learn → sometimes being right isn't about winning the argument, it's about finding a way to get the right outcome without creating friction

---

G — All Nighters (Backup Only)

Use for: Deliver Results (last resort)

Situation: I had an assignment deadline approaching and I was behind.

Action: I pulled two all nighters to get it done.

Result: Delivered on time. Not my proudest planning moment, but I learned to start earlier.

Only use if nothing else fits.

LP COVERAGE

#	Leadership Principle	Story	Fit
1	Customer Obsession	I — Community Events	✅
2	Ownership	A — Automation	✅
3	Invent and Simplify	A — Automation / E — Distributed System	✅
4	Are Right, A Lot	J — Distributed Systems Teammate	✅
5	Learn and Be Curious	D — Not Asking for Help	✅
6	Hire and Develop the Best	—	Skip
7	Insist on Highest Standards	F — SSO Access	✅
8	Think Big	H — Coding Community Contest	⚠️
9	Bias for Action	A — Automation	✅

#	Leadership Principle	Story	Fit
10	Frugality	A — Automation	✅
11	Earn Trust	C — Client Escalation	✅
12	Dive Deep	C — Client Escalation / E — Distributed System	✅
13	Have Backbone; Disagree and Commit	B — Testing Disagreement	✅
14	Deliver Results	B — Testing Disagreement / A — Automation	✅
15	Strive to be Earth's Best Employer	—	Skip
16	Success and Scale Bring Broad Responsibility	—	Skip

QUICK TRIGGER TABLE

If they ask...	Use story...
"Time you went above and beyond for a customer"	I — Community Events
"Time you took ownership"	A — Automation
"Time you simplified something complex"	A — Automation or E — Distributed System
"Time your judgment was right despite uncertainty"	J — Distributed Systems Teammate
"Time you learned something new"	D — Not Asking for Help or E — Distributed System
"Time you refused to lower the bar"	F — SSO Access
"Time you thought beyond what was asked"	H — Coding Community Contest
"Time you acted without waiting"	A — Automation
"Time you did more with less"	A — Automation
"Time you earned someone's trust"	C — Client Escalation
"Time you dug deep to find root cause"	C — Client Escalation or E — Distributed System
"Time you disagreed with a teammate"	B — Testing Disagreement
"Time you delivered under a tight deadline"	B — Testing Disagreement

If they ask...	Use story...
"Time you led something"	H — Coding Community Contest
"Time you failed or made a mistake"	C — Client Escalation
"Your weakness"	D — Not Asking for Help
"Most difficult challenge you're proud of"	E — Distributed File System
"Time your working style clashed with a teammate"	J — Distributed Systems Teammate

BOILERPLATE ANSWERS

"Tell me about yourself"

I'm Mahip, a Master's student in CS at Northeastern. Before grad school, I worked as a software engineer in India for a year on a product support team where I helped developers and client managers troubleshoot platform issues. Since starting my masters, I've built projects ranging from a distributed file storage system in C++ to a full stack learning management system. This semester I'm learning AWS and machine learning. I'm looking for a role where I can build things at scale and learn fast.

"Why Amazon?"

Two things. First, the scale. Amazon operates systems that serve millions of users and I want to experience what it takes to build and maintain software at that level. Second, the ownership culture. Engineers at Amazon are expected to own their work end to end, not just throw code over a wall. That matches how I like to work. When I built the automation script at my previous job, nobody told me to do it. I saw the problem, proposed the solution, and owned the whole thing. I want to be in an environment where that's the norm, not the exception.

"What are your future goals?"

Short term, I want to get solid industry experience, work on systems at scale, and learn from engineers who've been doing this longer than me. Long term, I want to own and lead technical projects end to end. Not just write code but understand the full picture. I don't have a rigid five year plan. I just want to keep putting myself in environments where I'm learning fast and building things that matter.

"Where do you see yourself in 5 years?"

I want to be a senior engineer who can take an ambiguous problem, break it down, and drive it to completion. Someone the team trusts to own critical pieces. I don't know exactly what domain yet, and I'm okay with that. Right now I'm focused on building a strong foundation and getting exposure to real world systems at scale.

"What is success to you?"

Knowing that the work I did made a real difference. Whether that's a tool that saved my team weeks of work, or helping a student submit their first CP solution. It's not about titles. It's about looking back and knowing I left things better than I found them. Personally, success is also growth. If I'm the same engineer a year from now, that's a failure.

"How do you handle multiple tasks?"

I prioritize based on urgency and impact. At IDX, I juggled JIRA tickets, QA checks, and SSO requests daily. I'd check tickets in the morning, block time for focused work, and handle quick requests in between. The key is being honest about what I can get done rather than saying yes to everything and delivering nothing well.

"What are your strengths?"

I take ownership without being told. At IDX, when my team needed a version report across 900+ websites, I proposed automating it and built the whole thing myself. I also hold firm on process. I managed SSO access and never let anyone bypass the JIRA ticket requirement, even when they tried.

"What is your weakness?"

Early in my career, I tried to figure out everything myself instead of asking for help. At my first job, I spent two weeks ramping up solo. My manager told me I wasn't learning fast enough and said the team has my back. I changed completely after that. I still try things myself first, but now if I'm stuck for 20-30 minutes, I reach out.

"Questions for the interviewer"

1. "What does a typical day look like for an intern on your team?"
2. "What separates interns who do really well from those who do okay?"
3. "What's something you wish you knew when you started at Amazon?"

PROJECT OVERVIEWS

Distributed File Storage System (C++)

We built a low level distributed file storage system with fault tolerance. It takes a file, breaks it into chunks, and stores them on different storage nodes. At retrieval, it collects chunks, reconstructs the file, verifies integrity by comparing hashes, and returns the final file. Why? I wanted to understand how distributed systems actually work under the hood.

Calendar Event Manager (Java)

Calendar event management system focused on clean design. MVC architecture, SOLID principles, design patterns like Factory, Adapter, and Command. Goal was easy extensibility. Add new features without touching existing tested code. 98% mutation test coverage with 200+ JUnit tests.

Kambaz LMS (React, Node.js, MongoDB)

Full stack replica of Canvas LMS. Course management, assignments, grading, real time updates via WebSockets. The project where I got comfortable with the full stack end to end.

PAST EXPERIENCE

Quick version: At IDX India, I was a software engineer on the product support team for about a year. Our company had a platform built on Drupal for investor relations websites. My team kept the product healthy. Resolving issues, handling upgrades, configuring backends, managing SSO access, and running QA checks before sites went live.

If they go deeper: I managed SSO access as a KPI, took JIRA tickets from developers and client managers, and was the gatekeeper for the QA checklist before sites went to production. I also got some exposure to React and Vue.js working alongside a team converting an internal tool from .NET.